

К aida64 можно обратиться из питона?

Да. К AIDA64 можно обращаться из Python, причём несколькими способами. Для твоего проекта AI-BreadBoard и будущих инструментов диагностики Windows это может быть полезно.

AIDA64 предоставляет официальные механизмы интеграции с внешними приложениями: Shared Memory, Windows Registry, WMI и командную строку.

 AIDA64 +1

1. Shared Memory — прямой доступ к датчикам

Наиболее интересный вариант для мониторинга в реальном времени.

AIDA64 публикует значения датчиков в именованной области разделяемой памяти Windows:

```
AIDA64_SensorValues
```

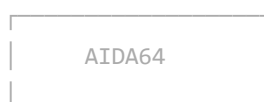
Из Python можно обратиться к этой памяти через Windows API (`OpenFileMappingW` , `MapViewOfFile`) с помощью `ctypes` .

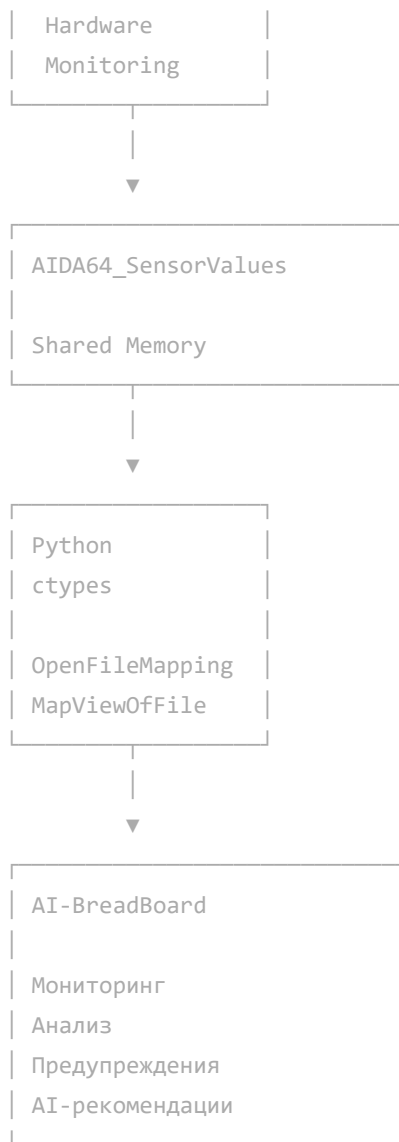
Какие данные доступны

- Температура CPU.
- Температура GPU.
- Температура материнской платы.
- Обороты вентиляторов.
- Напряжения.
- Загрузка CPU.
- Использование памяти.
- Другие показатели аппаратного мониторинга.

AIDA64 передаёт данные в строковом формате с XML-тегами, но это не полноценный XML-документ.  AIDA64 +1

Пример архитектуры





В настройках AIDA64 необходимо включить:

File → Preferences → Hardware Monitoring → External Applications → Enable shared memory

Официальное описание:  AIDA64

2. Windows Registry — проще всего для Python

AIDA64 может записывать значения датчиков в реестр:

```
HKEY_CURRENT_USER
├── Software
│   ├── FinalWire
│   │   ├── AIDA64
│   │   │   └── SensorValues
```

Из Python доступ осуществляется стандартным модулем `winreg`.

Пример

```
import winreg

def read_aida64_registry() -> dict[str, object]:
    '''Read AIDA64 sensor values from Windows Registry.

    :returns: Dictionary containing sensor values.
    :raises FileNotFoundError: If the registry key is unavailable.
    '''

    registry_path = r'Software\FinalWire\AIDA64\SensorValues'
    values = {}

    with winreg.OpenKey(
        winreg.HKEY_CURRENT_USER,
        registry_path,
    ) as key:
        index = 0

        while True:
            try:
                name, value, _ = winreg.EnumValue(key, index)
                values[name] = value
                index += 1
            except OSError:
                break

    return values
```

Особенность: AIDA64 удаляет ключ с показателями при закрытии программы.



3. WMI — интеграция с Windows-инструментами

AIDA64 может публиковать показания в пространстве WMI:

```
Root\WMI\AIDA64_SensorValues
```

Python может обращаться к WMI через соответствующие библиотеки Windows.

Например, можно использовать `wmi` или напрямую COM-интерфейсы через `pywin32`.

```
import wmi

def read_aida64_wmi() -> list[dict[str, object]]:
    '''Read sensor values from AIDA64 WMI namespace.

    :returns: List of WMI sensor records.
    '''

    connection = wmi.WMI(namespace=r'root\WMI')
    records = []

    for item in connection.AIDA64_SensorValues():
        records.append({
            'name': item.Name,
            'value': item.Value,
        })

    return records
```

⚠ Названия классов и свойств необходимо проверить по фактической структуре WMI-провайдера установленной версии AIDA64. Не следует считать этот пример универсальным без проверки.

Документация AIDA64:  AIDA64 +1

4. Командная строка — отчёты и диагностика

AIDA64 поддерживает запуск с параметрами командной строки.

Из Python можно использовать `subprocess`.

Что можно автоматизировать

Задача	Возможность
Создание отчёта о системе	Да
Сохранение в XML	Да
Сохранение в CSV	Да
Сохранение в HTML	Да
Отчёт об оборудовании	Да

Задача	Возможность
Программный запуск бенчмарков	Да
Автоматизированный аудит	Зависит от редакции
Удалённый аудит	Зависит от редакции

AIDA64 поддерживает параметры `/R` , `/CSV` , `/XML` , `/HW` , `/SUM` , `/BENCH` и другие.

Доступные параметры зависят от редакции программы.  AIDA64 +1

Пример Python

```
import subprocess
from pathlib import Path

def create_aida64_report(
    executable: str | Path,
    output_file: str | Path,
) -> None:
    '''Create a hardware report using AIDA64.

    :param executable: Path to AIDA64 executable.
    :param output_file: Destination report path.
    ...

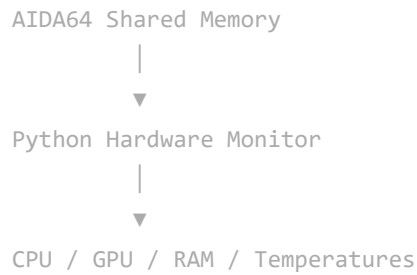
    subprocess.run(
        [
            str(executable),
            '/R',
            str(output_file),
            '/HW',
            '/XML',
            '/SILENT',
        ],
        check=True,
    )
```

Здесь важно сверить синтаксис параметров с документацией конкретной версии AIDA64.

Что я рекомендую для твоего проекта

Если задача — создать универсальный диагностический модуль Windows, я бы разделил интеграцию следующим образом.

Уровень 1. Непрерывный мониторинг

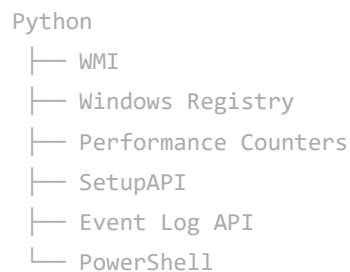


Подходит для:

- Отслеживания перегрева.
- Контроля нагрузки.
- Сбора телеметрии.
- Выявления аномалий.

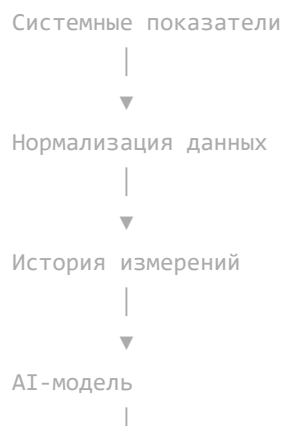
Уровень 2. Системная диагностика

Использовать собственные Windows API:



AIDA64 можно подключать как дополнительный источник аппаратных показателей, а не делать его обязательной зависимостью.

Уровень 3. AI-анализ



Например:

CPU достиг 95 °C при длительной нагрузке. Наблюдается снижение частоты.
Проверьте охлаждение, термопасту и воздушный поток корпуса.

При этом необходимо разделять фактически измеренные показатели и предположения модели.

Зажное ограничение



AIDA64 не является полноценным универсальным Windows API.

Он предоставляет интерфейсы для получения собственных измерений и отчётов. Для управления службами, драйверами, журналами, восстановлением системы и другими функциями Windows лучше использовать нативные механизмы операционной системы.

Официальная документация:

- [Внешние приложения AIDA64](#)
- [Параметры командной строки AIDA64](#)

Следующий практический шаг

Я бы начал с создания Python-модуля:

```
src/system_monitor/  
  __init__.py  
  aida64.py  
  sensors.py  
  performance.py  
  diagnostics.py
```

aida64.py будет отвечать за чтение Shared Memory, Registry и WMI, а остальные модули — за независимые Windows API.

Могу следующим сообщением [показать полноценную реализацию чтения AIDA64 Shared Memory через ctypes для Windows 10/11](#), с разбором получаемых показателей и обработкой ошибок.



А если я хочу использовать portable версию aida64?

Да, portable-версию AIDA64 можно использовать вместе с Python, и для твоего приложения это вполне подходящий вариант. Но есть несколько важных нюансов.

1. Как будет работать архитектура

Ты можешь хранить AIDA64 непосредственно в каталоге своего приложения:

```
AI-BreadBoard/
|
├─ app/
|   └─ ...
|
├─ tools/
|   └─ aida64/
|       ├── aida64.exe
|       ├── aida64.ini
|       └─ ...
|
├─ src/
|   └─ system_monitor/
|       └─ aida64.py
|
└─ config/
    └─ aida64.json
```

Python запускает AIDA64 при необходимости и получает данные через доступные интерфейсы.

```
Python Application
|
▼
AIDA64 Portable
|
├─ Shared Memory
├─ Registry
├─ WMI
└─ Reports
```

Установка AIDA64 в Windows не обязательно нужна, если используемая редакция и функции работают в portable-режиме.

2. Какие способы интеграции подходят для portable

Интерфейс	Portable	Особенности
Shared Memory	Да, при поддержке версией	Не требует установки отдельного драйвера
Registry	Да	Необходима настройка экспорта
WMI	Требует проверки	Зависит от реализации провайдера
Command Line	Да	Запуск через subprocess
Отчёты XML/CSV	Да	Удобно для разовых проверок
Прямой запуск EXE	Да	Указываешь путь к исполняемому файлу

Что я бы выбрал

Для твоего диагностического приложения:

- **Shared Memory** — мониторинг в реальном времени.
- **Command Line + XML** — получение подробного отчёта.
- **Собственные Windows API** — диагностика системы без зависимости от AIDA64.

3. Запуск portable AIDA64 из Python

Простейший вариант:

Python

```
from pathlib import Path
import subprocess

def start_aida64(executable: str | Path) -> subprocess.Popen:
    '''Start portable AIDA64.

    :param executable: Path to AIDA64 executable.
    :returns: Started AIDA64 process.
    :raises FileNotFoundError: If executable is missing.
    '''

    executable = Path(executable)
```

Run

```
if not executable.is_file():
    raise FileNotFoundError(
        f'AIDA64 executable not found: {executable}'
    )

process = subprocess.Popen(
    [str(executable)],
    cwd=executable.parent,
)

return process
```

Использование:

Python

Run

```
process = start_aida64(
    r'tools\aida64\aida64.exe'
)
```

`cwd` позволяет запускать программу из её собственного каталога, что важно для корректного поиска сопутствующих файлов.

4. Важный вопрос: лицензирование

Здесь есть принципиальное ограничение.

Portable ≠ свободная для распространения версия.

Если ты хочешь включить AIDA64 в свой AI-BreadBoard и распространять приложение другим пользователям, нужно проверить условия лицензии FinalWire.

Отдельно следует выяснить:

1. Можно ли запускать AIDA64 без установки.
2. Можно ли включать исполняемые файлы в сторонний пакет.
3. Разрешено ли автоматическое использование через Python.
4. Требуется ли лицензия для каждого компьютера.
5. Какие ограничения действуют для конкретной редакции.

Для личного использования и разработки это отдельный сценарий, но распространение требует проверки лицензии.

5. Можно ли сделать AIDA64 необязательной зависимостью?

Да, и это, на мой взгляд, полезная архитектура.

```
AI-BreadBoard
├── Native Windows Diagnostics
│   ├── CPU
│   ├── GPU
│   ├── RAM
│   ├── Disk
│   └── Drivers
└── Optional Providers
    ├── AIDA64
    ├── LibreHardwareMonitor
    ├── HWiNFO
    └── Direct Windows API
```

Пользователь может указать путь:

```
JSON

{
  "hardware_monitoring": {
    "enabled": true,
    "provider": "aida64",
    "executable": "tools/aida64/aida64.exe"
  }
}
```

При отсутствии AIDA64 приложение продолжает работать с собственными Windows API.

Мой практический совет

Если ты планируешь **portable-системный диагностический комплекс**, лучше не делать AIDA64 единственным источником данных.

Используй собственные API Windows как основной уровень, а AIDA64 подключай как дополнительный источник аппаратных датчиков.

Это позволит:

- Не зависеть от наличия AIDA64.
- Не привязывать базовую диагностику к сторонней лицензии.
- Добавлять другие аппаратные провайдеры.
- Использовать единый интерфейс данных для AI-модели.



Files, images, and data analysis are unavailable until usage resets at 8:56 AM. Continue chatting with text only, or upgrade for more access.

✦ [Try Plus free](#)

ChatGPT can make mistakes. Check important info.